Reinforcement Learning

Part 4: optimising the policy directly

Part 4 of 7. The policy gradient theorem, REINFORCE, actor-critic, TRPO and PPO.

Three lectures of doing it indirectly

Everything so far follows the same detour:

estimate values $\longrightarrow$ take $\arg\max_a$ $\longrightarrow$ get a policy

We never wanted the values. We wanted the behaviour. The values were a device for producing it.

So why not optimise the behaviour itself? Write the policy as $\pi_\theta(a|s)$, define $J(\theta)$ as the expected return, and do gradient ascent on it like every other model in this course.

There is one obstacle, and it is a real one: *the thing you are averaging over depends on the parameters you are differentiating*. This lecture removes that obstacle and then spends the rest of its time making the result usable.

Outline

Why bother

The policy gradient theorem

Making the gradient usable

Trust regions and PPO

Continuous control

Learn the behaviour, not the scoreboard

Three things $\arg\max_a Q$ simply cannot do.

Two ways to be an agent

Value-based

Learn $Q(s, a)$, then act greedily.

Good: sample-efficient, off-policy replay.

Bad: needs $\arg\max_a$, so actions must be few and discrete. Try that with a robot arm's continuous torques.

Policy-based

Learn $\pi_\theta(a|s)$ directly and improve it.

Good: continuous actions, naturally stochastic, optimises what you care about.

Bad: high variance, and normally on-policy – yesterday's data is stale.

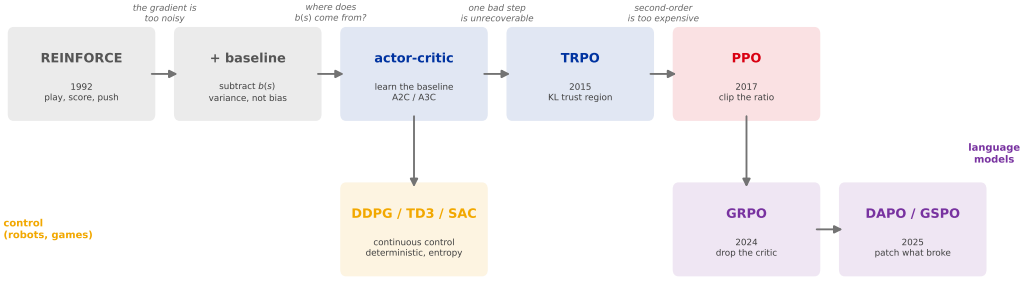
Three concrete failures of the value-based route:

- **Continuous actions.** $\arg\max_a$ over a real-valued torque is its own optimisation problem, at every single step.
- **Stochastic optima.** In rock-paper-scissors the best policy is uniformly random. A greedy policy cannot represent that.
- **Huge action spaces.** A language model's action space is the entire vocabulary at every token.

Where this lecture is going

Every one of these is REINFORCE plus one edit.

The edits do two jobs only: reduce the variance of the gradient, or stop the step being too big.



Keep this picture for the rest of the chapter. Lectures 6 and 7 do not introduce a new family – they walk further right along this one. **GRPO is PPO with one box removed.**

Differentiating an expectation you cannot differentiate

Six lines that make the whole family possible.

The obstacle, stated precisely

We want to maximise the expected return over trajectories the policy itself generates:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\big[R(\tau)\big] = \int P(\tau;\theta)\, R(\tau)\, d\tau$$

Why the usual move fails

In supervised learning the data is fixed and only the prediction depends on θ.

Here θ changes **which trajectories you see**. The distribution is the thing being optimised.

And $R(\tau)$ is a black box

The reward may be a game score, a unit test, or a human's opinion. There is no $\nabla_\theta R$. There may not even be a formula.

So: differentiate the *probability*, never the reward.

The log-derivative trick

One identity does all the work: $\nabla_\theta P = P\,\nabla_\theta \log P$ (just the chain rule on log, rearranged).

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta \int P(\tau;\theta) R(\tau)\, d\tau && R \text{ does not depend on } \theta \\
&= \int \nabla_\theta P(\tau;\theta)\, R(\tau)\, d\tau && \text{move the gradient inside} \\
&= \int P(\tau;\theta)\, \nabla_\theta \log P(\tau;\theta)\, R(\tau)\, d\tau && \text{the identity above} \\
&= \mathbb{E}_{\tau\sim\pi_\theta}\big[\, \nabla_\theta \log P(\tau;\theta)\, R(\tau) \,\big] && \textbf{an expectation again – so we can sample it}
\end{aligned}
$$

The trajectory probability factorises, so the dynamics drop out (**no model needed**), just as they did for importance sampling:

$$
\log P(\tau;\theta) = \cancel{\log p(s_0)} + \sum_t \Big[\log \pi_\theta(a_t|s_t) + \cancel{\log P(s_{t+1}|s_t, a_t)} \Big] \implies \nabla_\theta \log P = \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)
$$

What it says, and REINFORCE

$$\nabla_\theta J(\theta) \;=\; \mathbb{E}_{\pi_\theta}\big[\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t\big]$$

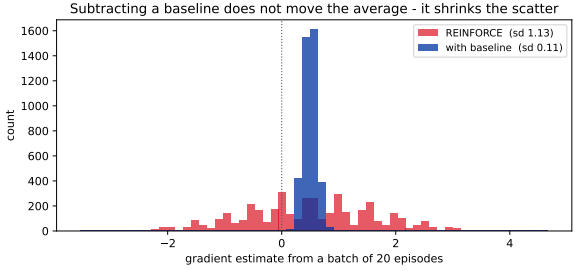
In words: take the actions you actually played, and adjust their probabilities in proportion to how well things went afterwards. Good outcome – make that action more likely. Bad outcome – less likely.

REINFORCE (Williams, 1992) is this used literally:

1. Run an episode with π_θ, storing every (s_t, a_t, r_t).
2. Compute the return G_t from each step onwards.
3. $\theta \leftarrow \theta + \alpha \sum_t G_t \nabla_\theta \log \pi_\theta(a_t|s_t)$.

Three lines, no model, no $\arg\max$, works with any reward you can compute. **And in this raw form it barely works.**

REINFORCE has a variance problem



Two actions, one state. Action A returns about 11, action B about 9 – so A is better and the gradient *should* point towards A.

Batches of 20 episodes, repeated 4000 times:

Plain REINFORCE points the **wrong way** 33.9% of the time. Sd 1.13.

With a baseline of 10: wrong way 0.0%, sd 0.11 – a 10× reduction, *same* average.

Why it works: both returns are positive, so plain REINFORCE pushes *both* actions up and only the sizes differ. Centring on the average makes the better action positive and the worse one negative – the signal stops fighting itself.

Same gradient, less noise

Baselines, advantages, and a dial you have already met.

Why subtracting a baseline is free

Replace G_t with $G_t - b(s_t)$ for any function b that does not depend on the action. The claim is that this changes the variance but **not** the expected gradient:

$$
\begin{aligned}
\mathbb{E}_{a\sim\pi}\left[\nabla_\theta \log \pi_\theta(a|s)\, b(s)\right] &= b(s) \sum_a \pi_\theta(a|s)\, \nabla_\theta \log \pi_\theta(a|s) && b(s) \text{ is a constant w.r.t. } a \\
&= b(s) \sum_a \nabla_\theta \pi_\theta(a|s) && \text{the identity, backwards} \\
&= b(s)\, \nabla_\theta \sum_a \pi_\theta(a|s) && \text{swap sum and gradient} \\
&= b(s)\, \nabla_\theta 1 \;=\; \mathbf{0} && \text{probabilities sum to one}
\end{aligned}
$$

A free lunch, and they are rare. You may subtract *anything* that does not depend on the action and the gradient stays correct in expectation. The best choice is the one that shrinks the variance most, which is roughly $b(s) = V(s)$.

Remember this proof. It is exactly why GRPO is allowed to use the mean reward of a group of answers as its baseline – lecture 7.

Advantage, and the actor-critic

$$A(s, a) = Q(s, a) - V(s)$$

"Was this action better or worse than what I normally do here?"

Actor

The policy π_θ. Chooses what to do. Trained by the policy gradient.

Critic

An estimate of $V_\phi(s)$. Provides the baseline. Trained by regression on returns.

The baseline from the previous slide, *learned* instead of assumed. It accepts a little bias in exchange for a large drop in variance – and it is the reason a value function reappears inside methods whose whole point was to avoid one.

Remember the critic. It is a second full-sized network. For a 70B language model that is 70B extra parameters to hold and train – and when you meet GRPO in lecture 7, its entire selling point is **throwing this away**.

GAE: the dial from lecture 2, again

How should the critic estimate the advantage? The same bias-variance dial as n-step returns:

Estimator	Bias	Variance
$\hat{A} = r_t + \gamma V(s_{t+1}) - V(s_t)$ (one step)	high (leans on V)	low
$\hat{A} = G_t - V(s_t)$ (full return)	none	high
GAE(λ): exponentially weighted average of all n-step versions	tunable	tunable

Generalised Advantage Estimation (Schulman et al., 2015)

$$\hat{A}_t^{\text{GAE}} = \sum_{l \geq 0} (\gamma\lambda)^l \, \delta_{t+l}, \qquad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

$\lambda = 0$ gives the one-step version; $\lambda = 1$ gives the full return. In practice $\lambda \approx 0.95$, and it is one of the few hyperparameters genuinely worth tuning.

Same idea as TD(λ), applied to advantages instead of values.

Do not take a step you cannot walk back

The single most important idea for everything that follows.

Why one bad step is fatal here and not in chapter 5

Supervised learning

Too large a learning rate ruins the model.

You lower it, reload the checkpoint, and rerun. **The dataset is still there.**

Reinforcement learning

Too large a step ruins the policy.

The ruined policy now **collects the next batch of data**. It explores badly, so it learns badly, so it explores worse.

This is the feedback loop that makes RL training feel unstable in a way supervised training does not. There is no fixed dataset to fall back on: **the data supply is downstream of the model's quality.**

So the goal is not just "take a good step". It is "take the biggest step you can while being sure you have not broken the thing that feeds you".

The surrogate objective, and a familiar ratio

There is a second problem, and the fix for it turns out to fix the first. After one gradient step the data you collected is *off-policy* – it came from π_{old}. Lecture 2 told us what to do about that:

$$L(\theta) = \mathbb{E}\Big[\underbrace{\frac{\pi_\theta(a|s)}{\pi_{\text{old}}(a|s)}}_{r_t(\theta)} \hat{A}_t \Big]$$

the importance ratio, cut down from a whole-trajectory product to **one step**

- At $\theta = \theta_{\text{old}}$ the ratio is 1 and this has the same gradient as plain policy gradient. So it is a legitimate stand-in.
- Away from θ_{old} it corrects for the mismatch – which means you can take **several** gradient steps on one batch of data instead of throwing it away.

And there is the trap. Nothing stops the optimiser making $r_t(\theta)$ enormous for one lucky action with a big advantage. The correction is only valid while the two policies are close

TRPO: put a fence around it

Trust Region Policy Optimization (Schulman et al., 2015) states the obvious fix as a constrained problem:

$$\max_{\theta} \; \mathbb{E}\big[\, r_t(\theta)\, \hat{A}_t \,\big] \qquad \text{subject to} \qquad \mathbb{E}\big[\, \mathrm{KL}(\pi_{\text{old}} \,\|\, \pi_\theta) \,\big] \le \delta$$

"Improve as much as you like, but do not move the distribution more than δ."

What it gets right

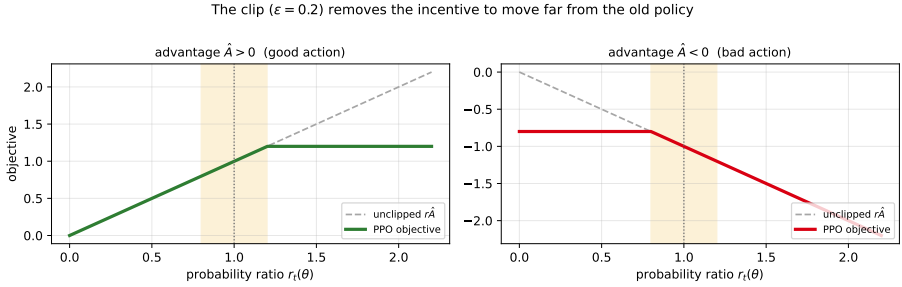
Monotonic improvement guarantees, and it genuinely stabilised policy gradients for the first time.

Why nobody uses it

Enforcing that constraint needs the Fisher information matrix and conjugate gradients. It is second-order, fiddly, and awkward to fit alongside dropout or parameter sharing.

The question that produced PPO: *can we get the fence with only first-order machinery?*

Proximal Policy Optimization: clip instead of constrain



$$L^{\text{CLIP}}(\theta) = \mathbb{E}\Big[\ \min\big(\ r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta),\ 1-\epsilon,\ 1+\epsilon)\ \hat{A}_t\ \big)\Big]$$

PPO – no constraint, no second derivatives, just a min and a clip, and it runs in any autodiff framework. Schulman et al., 2017.

Reading the clip properly

The min is the part people get wrong, so read both halves of the figure:

Good action, $\hat{A} > 0$

You want to raise its probability. The objective rises until $r_t = 1 + \epsilon$, then goes **flat**.

Past that point the gradient is zero: pushing further gains nothing.

Bad action, $\hat{A} < 0$

You want to lower its probability – and here the clip is *not* symmetric.

The min keeps the steeper branch, so an action that is much more likely than it should be still gets pushed down hard.

The asymmetry is deliberate. Removing the incentive to over-commit to a good action is safe. Removing the ability to escape a bad action would not be – so the clip lets you always run away from mistakes, and only limits your enthusiasm.

The full objective, term by term

$$L(\theta, \phi) = \underbrace{L^{\text{CLIP}}(\theta)}_{\text{(1) improve the policy}} - c_1 \underbrace{\left(V_\phi(s) - V^{\text{targ}}\right)^2}_{\text{(2) train the critic}} + c_2 \underbrace{\mathcal{H}\left[\pi_\theta(\cdot|s)\right]}_{\text{(3) stay curious}}$$

1. **The clipped surrogate.** Everything on the last two slides.
2. **The value loss.** The critic must be accurate or the advantages are garbage. Usually the same network body with two heads, which is why the two losses are added rather than optimised separately.
3. **The entropy bonus.** Rewards the policy for staying uncertain. Without it a policy that finds something that works will collapse onto it and stop exploring – and once entropy hits zero, no gradient can revive it.

Memorise the shape of this: clip + value + entropy. In lecture 6 you will see it again with a KL-to-reference penalty added, and in lecture 7 with term (2) deleted entirely.

PPO in practice

PPO's reputation comes from being hard to break, not from being subtle. The knobs that matter:

Knob	Typical	What goes wrong
clip ϵ	$0.1 - 0.2$	too big and the trust region stops meaning anything
epochs per batch	$3 - 10$	too many and π_θ drifts far from π_{old}, where the ratio is no longer trustworth
GAE λ	0.95	the bias-variance dial
entropy coef c_2	$0 - 0.01$	too small: entropy collapse. Too big: it never commits
advantage norm.	per batch	unnormalised advantages make the step size depend on the reward scale

What to watch while it trains: the KL between π_θ and π_{old}, the fraction of samples being clipped, and the policy entropy. If entropy is falling towards zero, the run is dying whatever the reward curve says. *This is the single most useful diagnostic in the chapter, and it is the same one used for reasoning models in lecture 7.*

The other branch

Where the actions are torques, not tokens.

DDPG, TD3, SAC

Method	Idea	The problem it fixes
Deep Deterministic Policy Gradient (**DDPG**, 2015)	A *deterministic* actor $\mu_\theta(s)$ outputs the action; the critic's gradient $\nabla_a Q$ tells it which way to move. Off-policy, with replay.	$\arg\max_a$ over continuous actions – the actor *is* the argmax, learned.
Twin Delayed DDPG (**TD3**, 2018)	Two critics and take the smaller; update the actor less often than the critic; smooth the target over nearby actions.	DDPG's overestimation and brittleness. *Twin critics are Double Q-learning again – third appearance.*
Soft Actor-Critic (**SAC**, 2018)	Maximise reward **plus entropy**: $\mathbb{E}[\sum_t r_t + \alpha\mathcal{H}(\pi(\cdot\|s_t))]$, with α tuned automatically.	Exploration and brittleness at once. Being deliberately uncertain is written into the objective rather than bolted on.

SAC is the default for robotics, PPO the default for everything else – and the reason is sample efficiency. SAC is off-policy and reuses a replay buffer; PPO throws its data away after a few epochs. When each sample costs a real robot movement, that gap decides the project.

Which one should you reach for?

Situation	Reach for	Why
Discrete actions, cheap simulator	DQN + Rainbow	replay makes it sample-efficient
Continuous actions, expensive samples	SAC	off-policy, entropy-regularised
Anything where stability matters more than efficiency	PPO	hard to break, few surprises
Huge discrete action space (tokens)	PPO / GRPO	no $\arg\max$, no critic over the vocabulary

An honest note on the last row. PPO became the standard for language models partly on merit and partly because it was what worked in 2017 when OpenAI wired up RLHF. Lecture 7 is largely the story of the field discovering which parts of PPO were load-bearing for text and which were inherited baggage.

Recap

- You cannot differentiate the reward, so differentiate the **log-probability of the action** instead and weight it by the return. That is the policy gradient theorem, and the environment dynamics cancel out of it.
- Raw REINFORCE points the wrong way **33.9%** of the time on a two-action toy problem. Every method after it is variance reduction or step-size control.
- Subtracting a **baseline** that does not depend on the action is provably free. Learning that baseline gives you the **actor-critic**; **GAE** is the dial controlling how much you trust it.
- In RL a ruined policy collects the next batch, so a step that is too large cannot be undone. **TRPO** fences this with a KL constraint; **PPO** gets the same effect by clipping the importance ratio, first-order and cheap.
- The PPO objective is **clip** + **value** + **entropy**. Watch KL, clip fraction and entropy while training – a collapsing entropy means a dying run.
- **SAC** for robots, **PPO** for stability and for tokens.

Next: everything so far has been reactive – one forward pass, one action. What happens if the agent is allowed to *think* before it moves? Tree search, AlphaZero, and what RL looks like when you cannot simulate anything at all.